



CSE461

Introduction to Robotics Lab

Lab No. : 05

Group : 06

Section : 04

Semester : Fall_2025

Group members :

Rifat Mahmud Tamim	21201787
Faishal Monir	22101235
Rubiya Karim	22101705
Rajin Ibna Rajuanur Rahman	22101717
Umma Salma Mim	22101870

Submitted Date: 03-12-2025

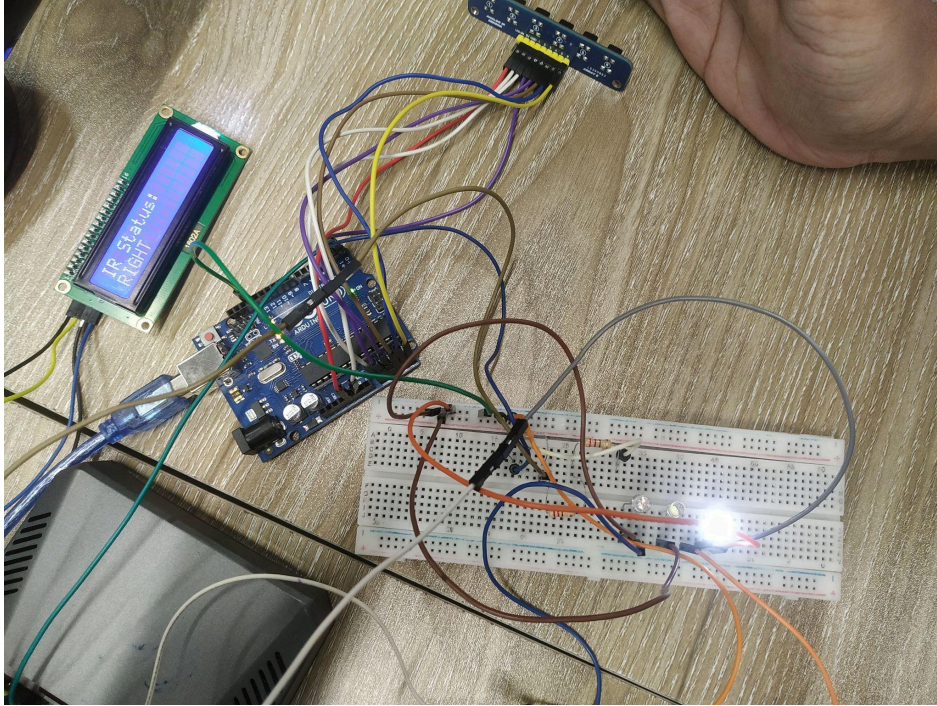
Submitted to - Aabrar Islam and Purba Tahia Hassan

1. Objectives: The objective of this lab is to interface an Arduino Uno and Raspberry Pi using the UART protocol to enable real-time transfer of IR sensor data. The lab aims to read analogue values from a 6-array IR sensor, send the processed output to the Raspberry Pi, and display the sensor status on an I2C LCD. Additionally, the experiment focuses on implementing a simple LED-based indication system on the Raspberry Pi and ensuring proper voltage-level compatibility between both devices. Through this activity, we developed skills in microcontroller communication, circuit design, and hardware software integration.

2. Equipment:

- Arduino Uno R3
- Raspberry Pi 4
- Analogue 6-array IR sensor
- I2C LCD Display (16x2)
- Resistor (220Ω)
- Breadboard
- Jumper wires
- LED

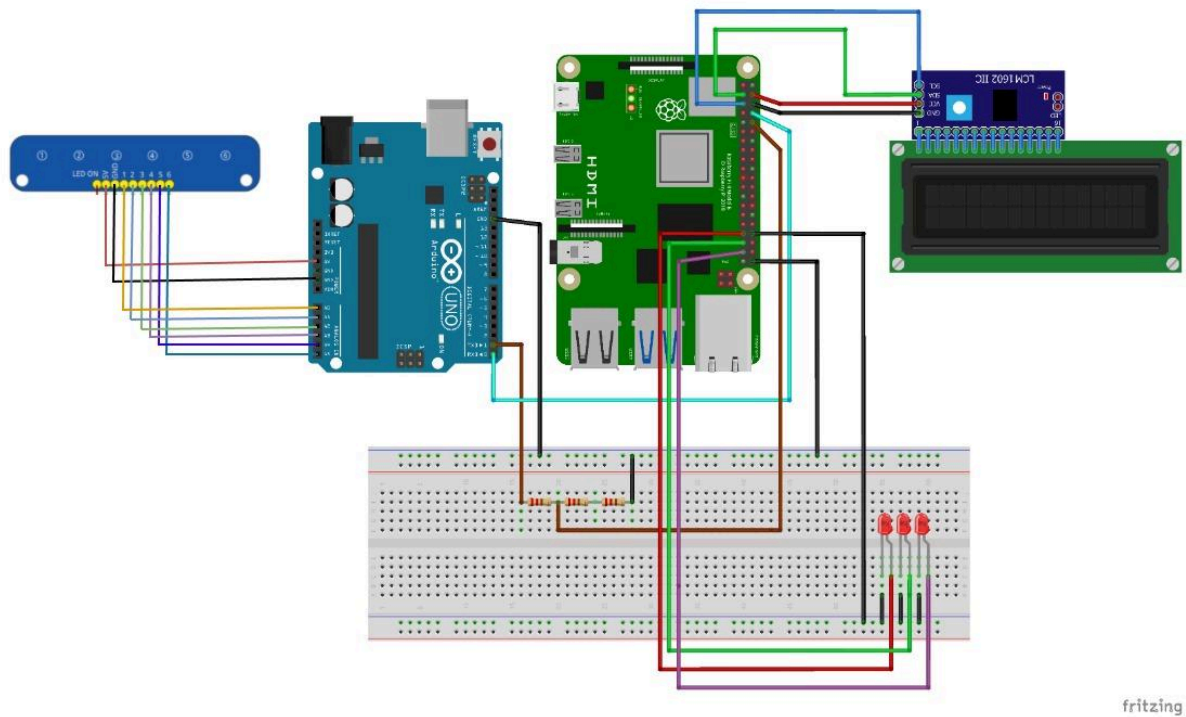
3. Experimental Setup:



In this experiment, the Arduino Uno was used to read values from the 6-array IR sensor and decide whether the system should display LEFT, RIGHT, or FORWARD. The Arduino sent these messages to the Raspberry Pi using UART communication.

On the Raspberry Pi, three LEDs were connected to GPIO pins 13, 19, and 26. These LEDs were used to show the direction sent by the Arduino. An I2C 16×2 LCD was also connected to the Raspberry Pi to display the sensor status. This setup shows how the Arduino and Raspberry Pi communicate with each other, and how the Raspberry Pi controls LEDs and an LCD based on the received sensor data.

Circuit Diagram:



4. Code:

❖ Arduino Code:

```
#define NUM_SENSORS 6
int sensorPins[NUM_SENSORS] = {A0, A1, A2, A3, A4, A5};
int sensorValues[NUM_SENSORS];

void setup() {
  Serial.begin(9600);
  for (int i = 0; i < NUM_SENSORS; i++) {
    pinMode(sensorPins[i], INPUT);
  }
}

void loop() {
  for (int i = 0; i < NUM_SENSORS; i++) {
    sensorValues[i] = analogRead(sensorPins[i]);
  }

  int threshold = 500;
  int binary[NUM_SENSORS];
  for (int i = 0; i < NUM_SENSORS; i++) {
    binary[i] = (sensorValues[i] > threshold) ? 0 : 1;
  }
}
```

```

    }

    if (binary[0] == 1 && binary[1] == 1 && binary[2] == 1 && binary[3] == 1 &&
    binary[4] == 1 && binary[5] == 1) {
        Serial.println("STOP");
    }
    else if (binary[0] == 0 && binary[1] == 0 && binary[2] == 0 && binary[3] == 0
    && binary[4] == 0 && binary[5] == 0) {
        Serial.println("FORWARD");
    }
    else if (binary[0] == 0 || binary[1] == 0) {
        Serial.println("LEFT");
    }
    else if (binary[4] == 0 || binary[5] == 0) {
        Serial.println("RIGHT");
    }
    else {
        Serial.println("UNCERTAIN");
    }

    delay(200);
}

```

❖ Raspberry Pi Code:

```

import serial
from rpi_lcd import LCD
from time import sleep
from gpiozero import LED, Button
import time

lcd = LCD()

led1 = LED(13)
led2 = LED(19)
led3 = LED(26)

ser = serial.Serial('/dev/ttyS0', 9600, timeout=1)

try:
    while True:
        if ser.in_waiting > 0:

            line = ser.readline().decode('utf-8',
errors='ignore').strip()

            lcd.clear()
            lcd.text("IR Status: ", 1)
            lcd.text(line, 2)

```

```

        if line=="LEFT":
            led1.on()
            led2.off()
            led3.off()
        elif line=="RIGHT":
            led1.off()
            led2.off()
            led3.on()
        elif line=="FORWARD":
            led1.on()
            led2.on()
            led3.on()
        else:
            led1.off()
            led2.off()
            led3.off()

except KeyboardInterrupt:
    lcd.clear()
    ser.close()
    print("Program terminated")

```

5. Results (Output of the experiment):

The experiment was completed successfully. The Arduino correctly detected the IR sensor readings and sent the direction messages (**LEFT**, **RIGHT**, or **FORWARD**) to the Raspberry Pi through UART. The Raspberry Pi then displayed the received message on the I2C LCD and controlled the LEDs based on the direction.

During execution, the Python program continuously read the incoming serial messages and updated the LCD and LEDs in real time. For example, when the sensor output indicated **LEFT**, only the left LED turned ON, when **RIGHT** was received, the right LED turned ON. For **FORWARD**, all LEDs turned ON together. For any message outside these three (such as *STOP* or *UNCERTAIN*), all LEDs remained OFF.

Sensor Status	LED Output
Right	Right LED ON
Left	Left LED ON
Forward	All LEDs ON

These results confirm that the Raspberry Pi successfully received the sensor data from the Arduino, displayed the status on the LCD, and controlled the LEDs correctly through its GPIO pins.

6. Discussions/Answers:

This experiment demonstrated how the Raspberry Pi can receive direction messages from an Arduino using the UART protocol and then control LEDs and an LCD screen based on the received data. The results showed that the Raspberry Pi successfully read the IR sensor status sent by the Arduino and updated the I2C LCD in real time. The LEDs responded correctly according to the message: the left LED turned on for LEFT, the right LED turned on for RIGHT, and all LEDs turned on for FORWARD, while they remained off for any unknown or uncertain message. This confirms that both the input signals from the Arduino and the output control on the Raspberry Pi were functioning properly. During the experiment, some issues were encountered, such as inconsistent serial readings when the wires were loose or the UART connection was not properly grounded. After fixing the wiring, the communication became stable. Overall, the experiment was effective and helped in understanding UART communication, GPIO control, and Python programming on the Raspberry Pi, along with how sensor data can be used to drive real-time hardware actions.